

6.3 子集和数问题

- 已知 n 个不同的正数(也称为权) $w_i, 1 \leq i \leq n$, 以及和数为 M 。要求找出 w_i 的和数是 M 的所有子集。
- 例如, 若 $n=4$, $(w_1, w_2, w_3, w_4)=(11, 13, 24, 7)$, $M=31$, 则满足要求的子集是 $(11, 13, 7)$ 和 $(24, 7)$ 。
- 如果通过给出其和为 M 的那些 w_i 的下标来表示解向量, 比直接用这些 w_i 表示解向量更为方便。因此这两个解就由向量 $(1, 2, 4)$ 和 $(3, 4)$ 所描述。
- **显式约束条件** 要求 $x_i \in \{j \mid j \text{ 是 } w_i \text{ 的下标值}, 1 \leq j \leq n\}$ 。
- **隐式约束条件** 则要求没有两个 x_i 是相同的且相应的 w_i 和数等于 M 。
- 为了避免产生同一个子集的重复情况 (例如: $(1, 2, 4)$ 和 $(1, 4, 2)$ 表示同一个子集), **附加另一个隐式约束条件**: $x_i \leq x_{i+1},$

6.3 子集和数问题

- 已知 n 个不同的正数(也称为权) w_i 要求找出 w_i 的和数是 M 的所有子集
- 例如, 若 $n=4$, $(w_1, w_2, w_3, w_4)=(11, 13, 24, 7)$, $M=31$, 则满足要求的子集是 $(11, 13, 7)$ 和 $(24, 7)$ 。
- 如果通过给出其和为 M 的那些 w_i 的下标来表示解向量, 比直接用这些 w_i 表示解向量更为方便。因此这两个解就由向量 $(1, 2, 4)$ 和 $(3, 4)$ 所描述。
- 显式约束条件** 要求 $x_i \in \{j \mid j \text{ 是 } w_i \text{ 的下标值}, 1 \leq j \leq n\}$ 。
- 隐式约束条件** 则要求没有两个 x_i 是相同的且相应的 w_i 和数等于 M 。
- 为了避免产生同一个子集的重复情况 (例如: $(1, 2, 4)$ 和 $(1, 4, 2)$ 表示同一个子集), **附加另一个隐式约束条件**: $x_i \leq x_{i+1}$,

11	13	24	7
1	2	3	4

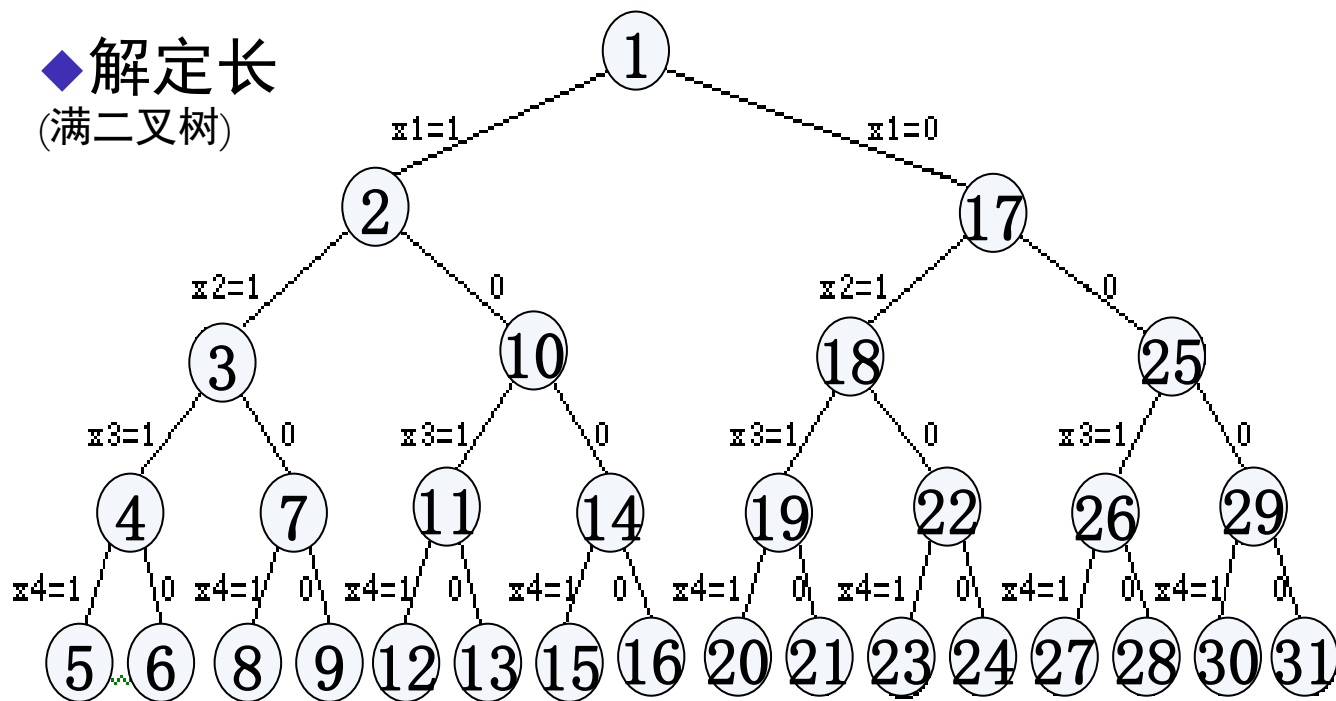
- **问题：**给定由 n 个不同正数组成的集合 $W=\{w_i\}$ ，和数 M ，求 W 中所有和等于 M 的子集的集合；
- **问题状态：**可以设想， W 中的每个数有选取和不选取两种可能， W 的一个子集即为一个问题状态，可以表述为对每个元素选取或不选取的一个组合。
- **问题状态空间树：**由空集开始，依次对每个元素进行选取和不选取两种选择，就可以得到 W 的全部子集，选取过程即可构成一棵状态空间树。
- 一个问题的解可以**有多种表示形式**，而这些表示形式都使得所有的解是满足某些约束条件的多元组。
 - (1) 每一个解的子集由这样一个 **n -元组 $(x_1, \dots, x_n)$ 所表示**，它使得 $x_i \in \{0, 1\}, 1 \leq i \leq n$ 。如果没有选择 w_i ，则 $x_i=0$ ；否则 $x_i=1$ 。
——**解定长法：使用大小固定的元组表示所有的解。**
 - (2) 每一个解的子集中 n -元组的构成是 $x_i \in \{w_i | 1 \leq i \leq n\}$ 。即如果选择 w_i ，则 $x_i=w_i$ ；否则 x_i 为空。
——**解不定长：使用大小不固定的元组表示所有的解。**

子集和数问题的状态空间树

如：设 $W=\{11, 13, 24, 7\}$ ，则状态空间树如下：

◆解定长

(满二叉树)



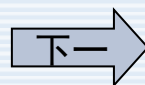
- 按照回溯法思想，从状态树的根结点出发，做深度优先搜索；根结点对应空元组 $\{\}$ 。

$\{11, 13, 7\} \Rightarrow \{1, 2, 4\} \Rightarrow \{1, 1, 0, 1\},$
 $\{24, 7\} \Rightarrow \{3, 4\} \Rightarrow \{0, 0, 1, 1\}$

• 对应于元组大小固定的列式表示。由 i 级到 $i+1$ 级结点的那些边用 x_i 的值来标记， x_i 的值为或者为1或者为0。

• 由根到叶结点的所有路径确定了解空间，根的左子树确定包含 w_1 的所有子集，而根的右子树则确定不包含 w_1 的所有子集。

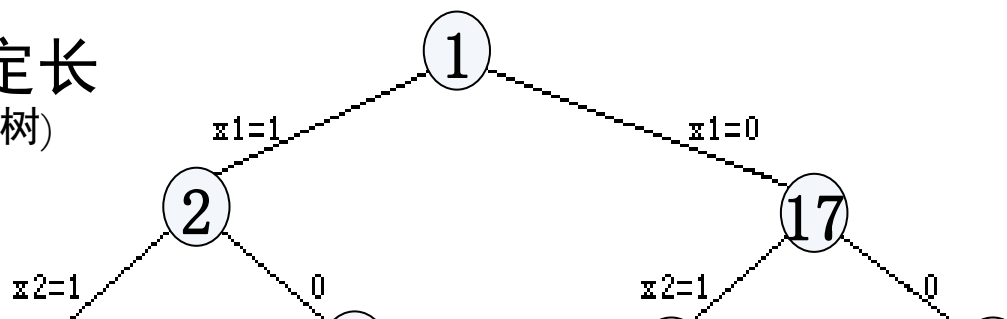
• 有 2^4 个叶结点，所以表示有16个可能的元组。



子集和数问题的状态空间树

如：设 $W=\{11,13,24,7\}$ ，则状态空间树如下：

◆解定长
(满二叉树)



•对应于元组大小固定的列式表示。由 i 级到 $i+1$ 级结点的那些边用 x_i 的值来标记， x_i 的值为或者

为便于计算，将 W 中的正数按**从小到大**排序；

当在某一状态 A 下，依次尝试加入和不加入正数 w_i ，

若 $\sum A + w_i > M$ ，则可停止对该结点的搜索；

若 $\sum A + \sum (w_i \cdots w_n) < M$ ，则也可停止对该结点的搜索；

(5)(6)(8)(9)(12)(13)(15)(16)(20)(21)(23)(24)(27)(28)(30)(31)

原始 $\{11,13,7\} \Rightarrow \{1,2,4\} \Rightarrow \{1,1,0,1\}$,

$\{24,7\} \Rightarrow \{3,4\} \Rightarrow \{0,0,1,1\}$

排序 $\{7,11,13\} \Rightarrow \{4,1,2\} \Rightarrow \{1,1,1,0\}$,

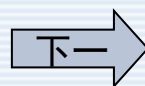
$\{7,24\} \Rightarrow \{4,3\} \Rightarrow \{1,0,0,1\}$

0。

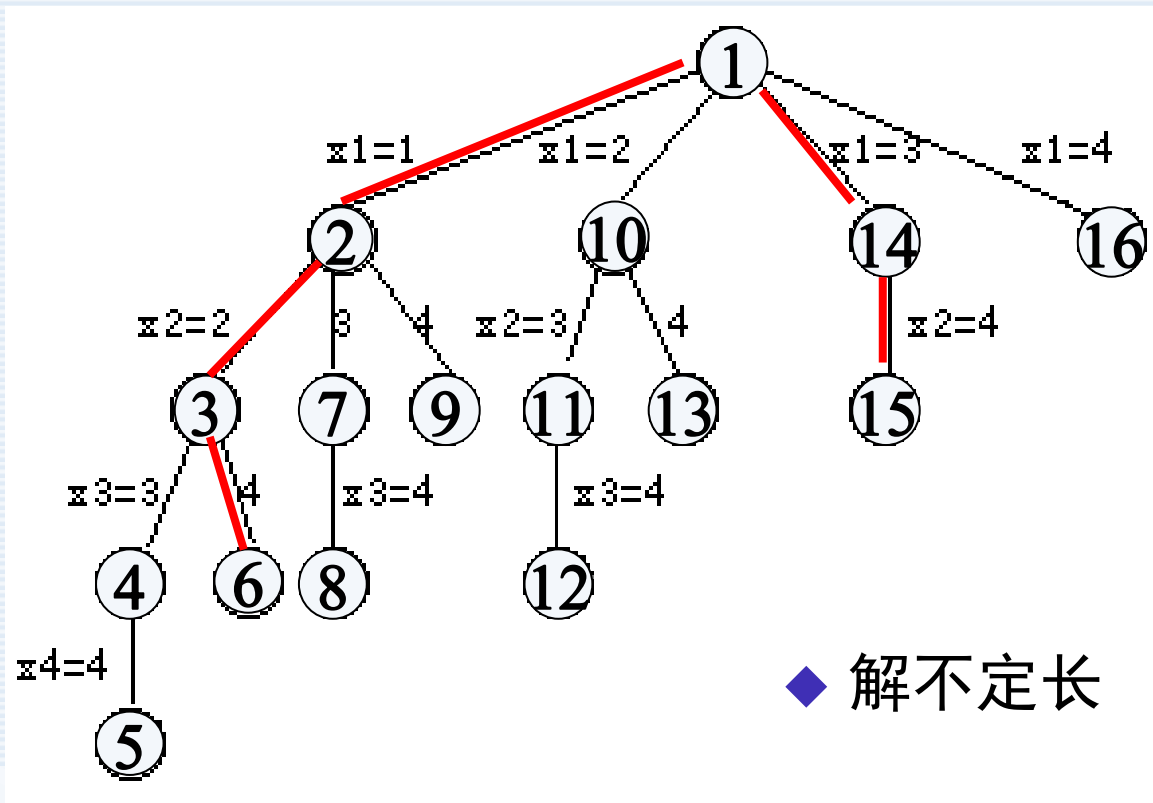
结点的所
定了这个
根的左子

例确定包含 w_1 的所有子集，而根的右子树则确定不包含 w_1 的所有子集。

有 2^4 个叶结点，所以表示有 16 个可能的元组。



子集和数问题的状态空间树

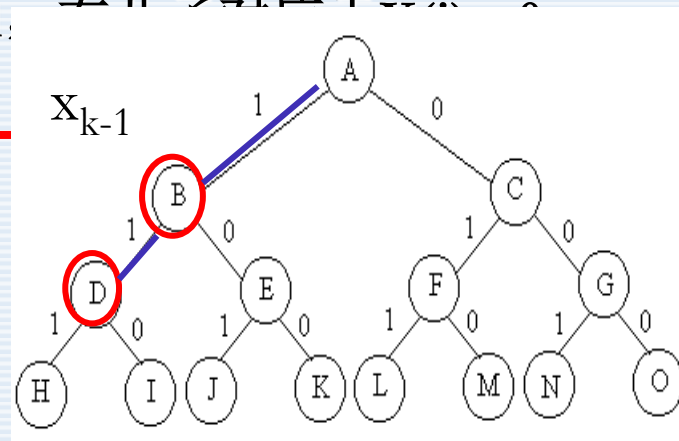


$\{11, 13, 7\} \Rightarrow \{1, 2, 4\} \Rightarrow \{1, 1, 0, 1\},$
 $\{24, 7\} \Rightarrow \{3, 4\} \Rightarrow \{0, 0, 1, 1\}$

• 对应于元组大小可变的列式表示，树边的标记方式是，由*i*级到*i+1*级的一条边用*x_i*来表示，在每一个结点处，解空间被分成一些子解空间，解空间则由树中的根结点到任何结点的所有路径所确定。

• 最左子树确定了包含 w_1 的所有子集，下一棵子树则确定了包含 w_2 但不包含 w_1 的所有子集。

- 前面已经说明过如何用大小固定或变化的元组来表示这个问题。 **本节将利用大小固定的元组来研究回溯解法，并解决这一问题。**
- 对于i级上的一个结点，其左儿子对应于 $X(i) = 1$ 。为便于计算，将W中的正数按**从小到大**排序；



• 限界函数

– 当且仅当 $\sum_{i=1}^k W(i)X(i) + \sum_{i=k+1}^n W(i) \geq M$

时， $B(X(1), X(2), \dots, X(k)) = \text{True}$

– 当W(i)以递增次序排列，则限界函数可强化：

– 若 $\sum_{i=1}^k W(i)X(i) + W(k+1) > M$ ，则 $X(1), X(2), \dots, X(k)$ 就不能导致一个答案结点。

- 因此将要使用的限界函数是： $B_k(X(1), X(2), \dots, X(k)) = \text{True}$ ，当且仅当

$$\sum_{i=1}^{k-1} W(i)X(i) + W(k) + W(k+1) \leq M$$

$$\sum_{i=1}^{k-1} W(i)X(i) + 0 + W(k+1) \leq M$$

$$\sum_{i=1}^{k-1} W(i)X(i) + \sum_{i=k}^n W(i) - W(k) \geq M$$

$$\sum_{i=1}^n W(i)X(i) \geq M$$

$$\sum_{i=1}^k W(i)X(i) + \sum_{i=k+1}^n W(i) \geq M$$

$$\sum_{i=1}^{k-1} W(i)X(i) + \sum_{i=k}^n W(i) \geq M$$

$$\sum_{i=1}^{k-1} W(i)X(i) + W(k) + \sum_{i=k+1}^n W(i) \geq M$$

$$\sum_{i=1}^k W(i)X(i) + W(k+1) + \sum_{i=k+2}^n W(i) \geq M$$

$$\sum_{i=1}^k W(i)X(i) + W(k+1) > M$$

所以：要满足

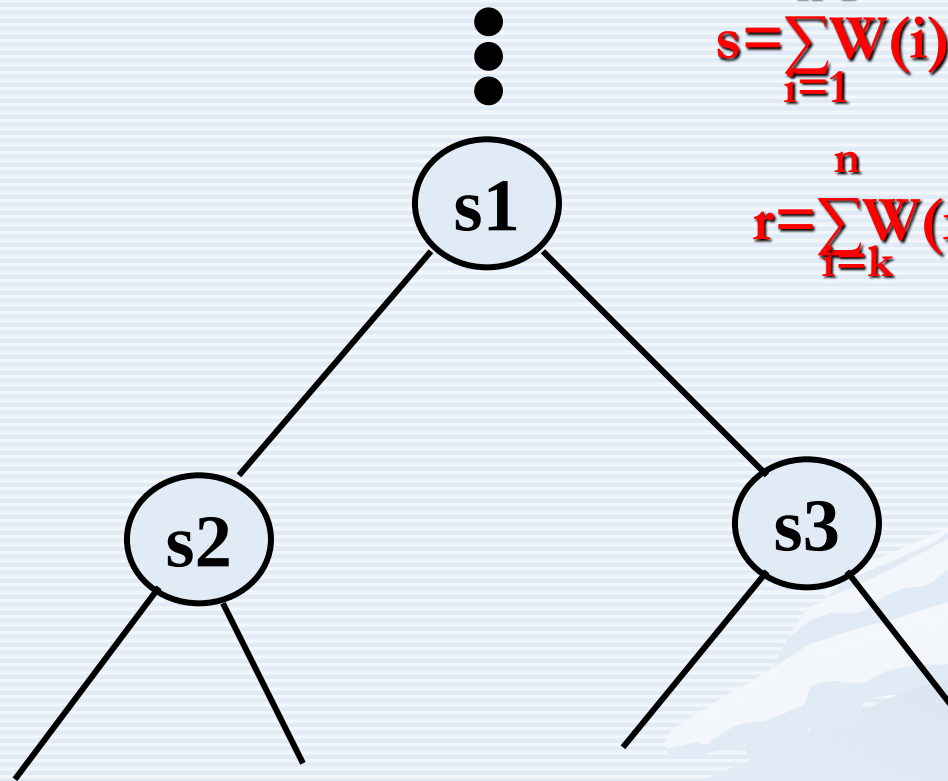
$$\sum_{i=1}^{k-1} W(i)X(i) + W(k) + W(k+1) \leq M$$

总和要大于M

对权值排序

满足 $W(k) < W(k+1)$

$s = \sum_{i=1}^{k-1} W(i)X(i)$: 子集的和
 k : 当前层数
 $r = \sum_{i=k}^n W(i)$ 剩余元素之和



B_k 为真

$$\sum_{i=1}^{k-1} W(i)X(i) + W(k) + W(k+1) \leq M$$

$$s + w_k + w_{k+1} \leq M$$

B_k 为假

$$\sum_{i=1}^{k-1} W(i)X(i) + 0 + W(k+1) < M$$

$$\sum_{i=1}^{k-1} W(i)X(i) + \sum_{i=k}^n W(i) - W(k) \geq M$$

$$(s + w_{k+1} \leq M)$$

$$(s + r - w_k \geq M)$$

可以约定：

$(x_1, x_2, \dots, x_{k-1}, 1)$ 和 $(x_1, x_2, \dots, x_{k-1}, 0)$ 所对应的状态结点分别是 $(x_1, x_2, \dots, x_{k-1})$ 所对应的状态结点左儿子和右儿子。
对于 i 级上的一个结点，其左儿子对应于 $X(i)=1$ ，右儿子对应于 $X(i)=0$ 。

$$s = \sum_{i=1}^{k-1} W(i)X(i) : \text{子集的和}$$

k : 当前层数

$$r = \sum_{i=k}^n W(i) \text{ 剩余元素之和}$$



B_k 为真

$$\sum_{i=1}^{k-1} W(i)X(i) + W(k) + W(k+1) \leq M$$

$$s + w_k + w_{k+1} \leq M$$

B_k 为假

$$\sum_{i=1}^{k-1} W(i)X(i) + 0 + W(k+1) < M$$

$$\sum_{i=1}^{k-1} W(i)X(i) + \sum_{i=k}^n W(i) - W(k) \geq M$$

$$(s + w_{k+1} \leq M)$$

$$(s + r - w_k \geq M)$$

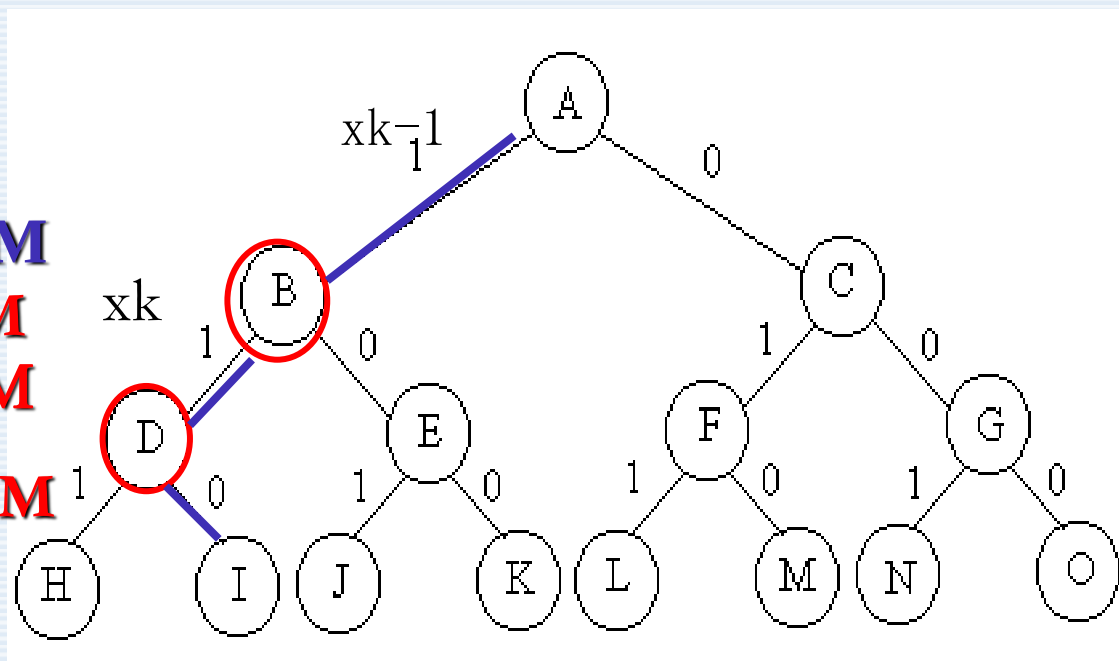
$$\sum_{i=1}^k W(i)X(i) + W(k+1) > M$$

$$\sum_{i=1}^k W(i)X(i) + \sum_{i=k+1}^n W(i) \geq M$$

$$\sum_{i=1}^k W(i)X(i) = M$$

$< M$
 $> M$

$$\sum_{i=1}^{k-1} W(i)X(i) + W(k) > M$$



可以约定：

$(x_1, x_2, \dots, x_{k-1}, 1)$ 和 $(x_1, x_2, \dots, x_{k-1}, 0)$ 所对应的状态结点分别是 $(x_1, x_2, \dots, x_{k-1})$ 所对应的状态结点左儿子和右儿子。对于 i 级上的一个结点，其左儿子对应于 $X(i)=1$ ，右儿子对应于 $X(i)=0$ 。

$$\sum_{i=1}^k W(i)X(i) + W(k+1) > M$$

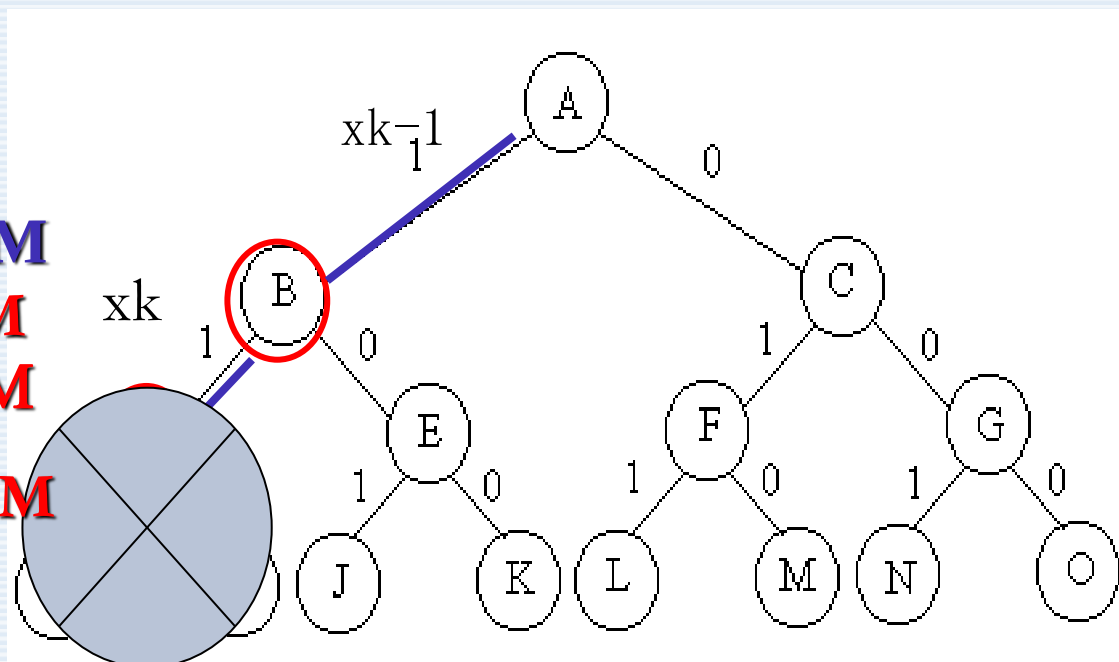
$$\sum_{i=1}^k W(i)X(i) + \sum_{i=k+1}^n W(i) \geq M$$

$$\sum_{i=1}^k W(i)X(i) < M$$

$$\sum_{i=1}^k W(i)X(i) = M$$

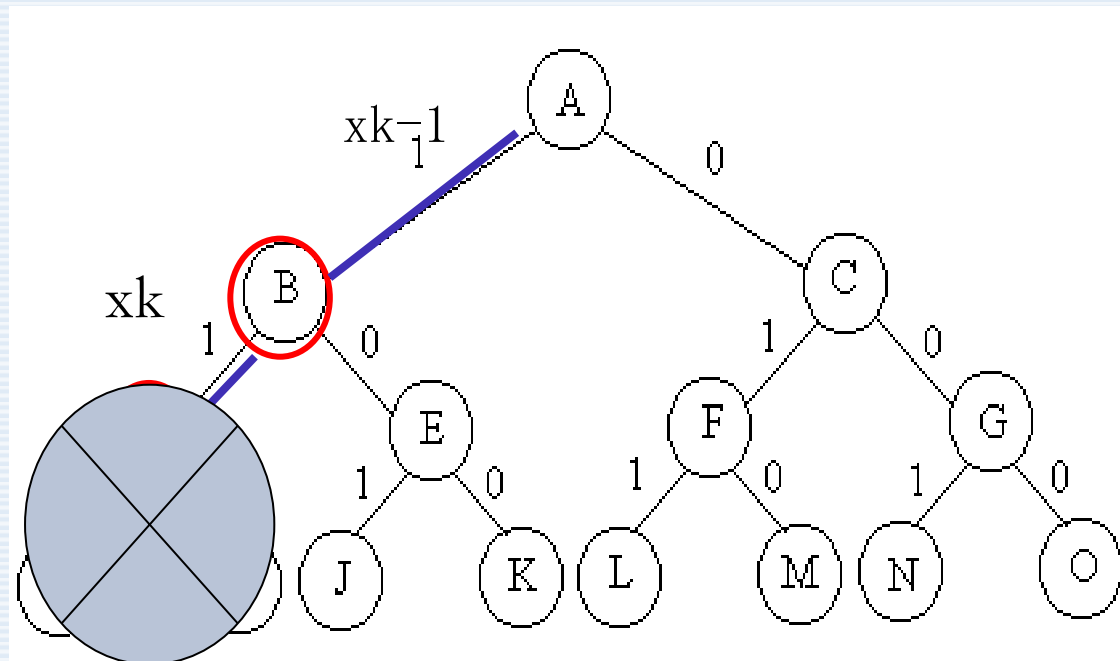
$$\sum_{i=1}^k W(i)X(i) > M$$

$$\sum_{i=1}^{k-1} W(i)X(i) + W(k) > M$$



可以约定：

$(x_1, x_2, \dots, x_{k-1}, 1)$ 和 $(x_1, x_2, \dots, x_{k-1}, 0)$ 所对应的状态结点分别是 $(x_1, x_2, \dots, x_{k-1})$ 所对应的状态结点左儿子和右儿子。对于 i 级上的一个结点，其左儿子对应于 $X(i)=1$ ，右儿子对应于 $X(i)=0$ 。



$$\sum_{i=1}^k W(i)X(i) < M$$

$$\sum_{i=1}^{k-1} W(i)X(i) + W(k) > M$$

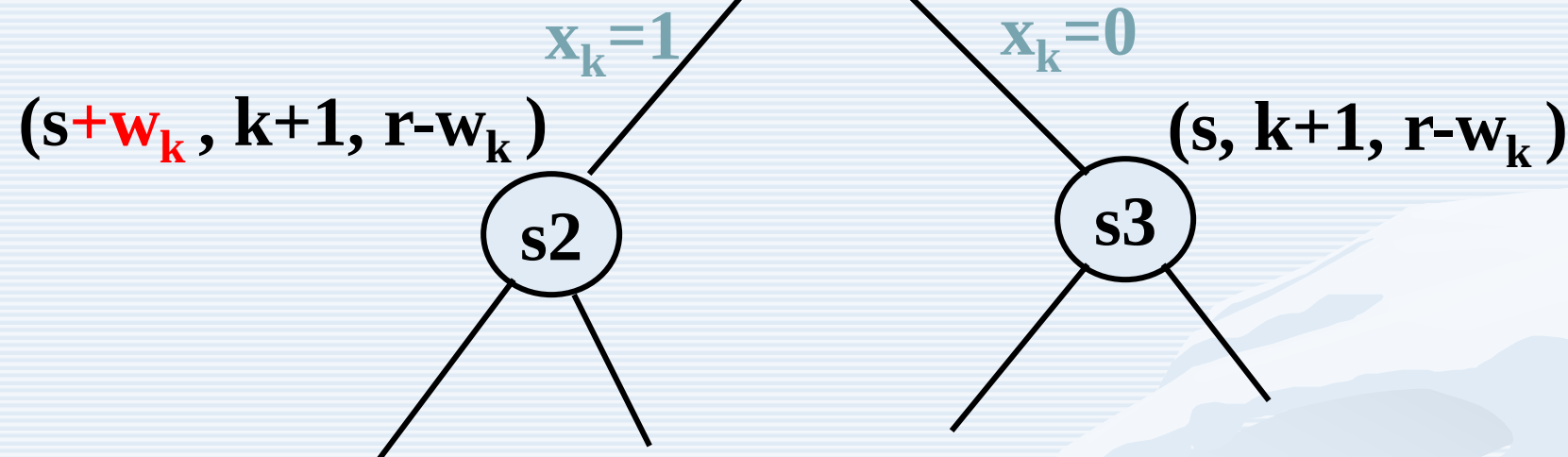
$$\sum_{i=1}^{k-1} W(i)X(i) + W(k) + W(k+1) \leq M$$

状态空间树的产生: (算法框架)

$x_{k-1} = \cdot$



$s = \sum_{j=1}^{k-1} W(j)X(j)$: 子集的和
 k : 当前层数
 $r = \sum_{i=k}^n W(i)$ 剩余元素之和



对于当前结点s1,

若s2是解结点, 则: (1)输出解; (2)考虑转向s3;

否则: (1)考虑转向s2; ●●●●●;

(2)考虑转向s3

为便于计算, 将W中的正数按从小到大排序;
 当在某一状态A下, 依次尝试加入和不加入正数 w_i ,
 若 $\sum A + w_i > M$, 则可停止对该结点的搜索;
 若 $\sum A + \sum (w_i \square w_n) < M$, 则也可停止对该结点的搜索;

五、“子集和”问题递归算法

$$r = \sum_{i=1}^n W(i) \quad x_{k-1} = \begin{matrix} \bullet \\ \bullet \\ \bullet \end{matrix} \quad s = \sum_{j=1}^{k-1} W(j) X(j)$$

Procedure SUMOFSUB(s, k, r);

{初始: $s = 0$; $r = \sum_{i=1}^n w_i$; $k = 1$ }

$x_k = 1$;

if $s + w_k = M$

then 输出解 $(x_1, x_2, \dots, x_n)$

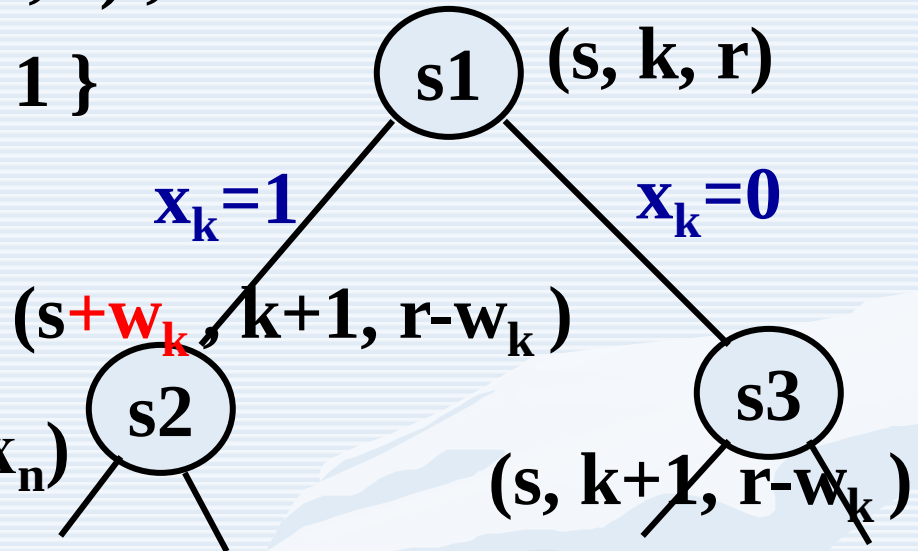
else

if $s + w_k + w_{k+1} \leq M$ //沿 $x_k=1$ 分枝延伸

then SUMOFSUB($s+w_k$, $k+1$, $r-w_k$);

if $(s + r - w_k \geq M)$ and $(s + w_{k+1} \leq M)$ //沿 $x_k=0$ 分枝延伸

then [$x_k = 0$; SUMOFSUB(s , $k+1$, $r-w_k$)]



算法6.6：子集和数问题的递归算法

Procedure SUMOFSUB(s,k,r)

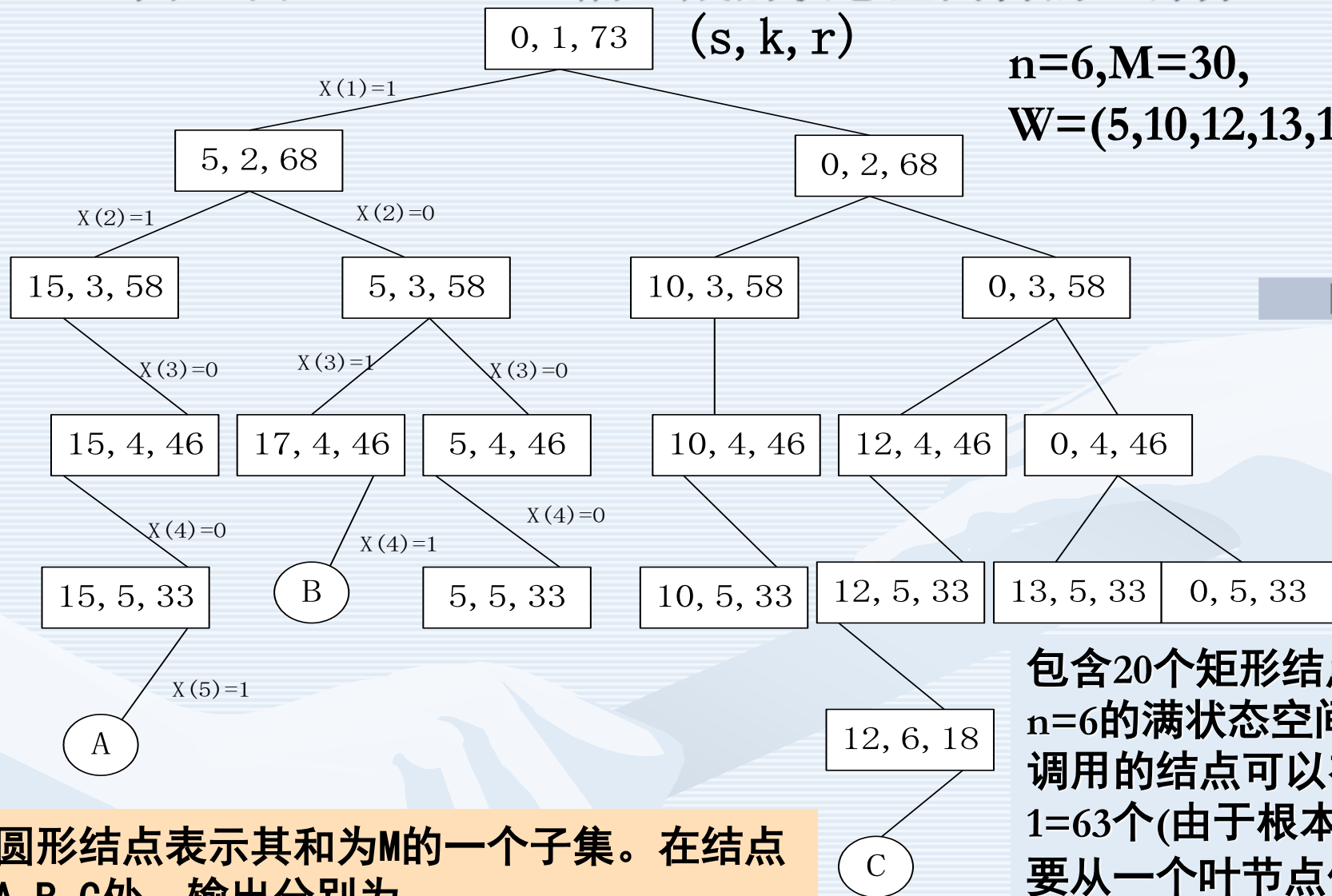
//找 $W(1:n)$ 中的和数为 M 的所有子集。进入此过程时 $X(1)$,
 $X(2),\dots,X(k)$ 的值已确定。 $S = \sum_{j=1}^{k-1} W(j)X(j)$,且 $r = \sum_{j=k}^n W(j)$ 。
这些 $W(j)$ 按非降次序排列。假定 $W(1) \leq M$, $\sum_{i=1}^n W(i) \geq M$ //

global integer M,n;global real W(1:n);global boolean X(1:n)
real r,s;integer k,j;
//生成左儿子。注意由于 $B_{k-1} = \text{true}$, 因此 $s + W(k) \leq M$ //
 $X(k) = 1$
if $s + W(k) = M$ then //子集找到//
 print(X(j),j=1 to k)
//此处由于 $W(j) > 0, 1 \leq j \leq n$,因此不存在递归调用//

算法6.6: 子集和数问题的递归算法

```
else  if  $s + W(k) + W(k+1) \leq M$  then  //  $B_k = \text{true}$  //  
      call SUMOFSUB( $s + W(k)$ ,  $k+1$ ,  $r - W(k)$ )  
    endif  
endif  
  
//生成右儿子和计算 $B_k$ 的值//  
if  $s + r - W(k) \geq M$  and  $s + W(k+1) \leq M$  //  $B_k = \text{true}$  //  
then  $X(k) = 0$   
    call SUMOFSUB( $s$ ,  $k+1$ ,  $r - W(k)$ )  
Endif  
End SUMOFSUB
```

图6.9由SUMOFSUB所生成的状态空间树的一部分



圆形结点表示其和为M的一个子集。在结点A, B, C处, 输出分别为
(1, 1, 0, 0, 1) (1, 0, 1, 1) 和 (0, 0, 1, 0, 0, 1)。

包含20个矩形结点, 而
 $n=6$ 的满状态空间树中
调用的结点可以有 $2^6-1=63$ 个(由于根本不需
要从一个叶节点作出调
用, 因此这一计数排除
了64个叶结点)

小结：

- (1) 回溯法的基本思想
- (2) 相关概念
- (3) 效率分析
- (4) 典型实例

作业：

复习回溯法， n 皇后问题，子集和数问题，23题

上交：P215: 1, 8, 9

上机：3, 4, 给出算法6.6的C语言求解程序。

自选：14, 18